Reproducible code for the NutNet multidimensional stability project

Step 1: Prepare and curate the input data

Qiang Yang

2026-06-22

Overview

This document prepares the input data for the NutNet multidimensional stability analyses. Starting from the raw Nutrient Network (NutNet) plant cover, aboveground biomass, and soil datasets, it applies the predefined inclusion criteria used in the manuscript, harmonizes plant taxonomic names within sites, merges cover and biomass records at the plot level, and appends baseline plot characteristics and site coordinates. The final output is the object df.tidy, which forms the basis for all subsequent stability calculations and statistical analyses.

The workflow retains only live vascular plant records, requires uninterrupted annual observations from treatment year 0 to treatment year 4, retains only blocks with an appropriate control comparison, and restricts the analytical dataset to unmanipulated control plots and single-nutrient addition treatments (nitrogen, phosphorus, and potassium). These procedures correspond to the sampling strategy described in the manuscript and Reporting Summary.

The cleaned datasets (df.tidy.Rdata) generated by this script are archived in this repository and are sufficient to reproduce all analyses, figures, and tables presented in the manuscript. Running this script is therefore not necessary to reproduce the main findings of the study. However, provided that the original NutNet data are available, this workflow can be used to regenerate the cleaned analytical datasets from the raw data. To fully reproduce the data-cleaning workflow implemented in this section, readers should obtain the raw data for the NutNet sites included in Supplementary Table 1. The code is extensively annotated to facilitate transparency, understanding, and reproducibility.

Output from this step

This script generates the following main output files:

- `df.tidy.Rdata`: merged plot-level dataset containing cover and biomass time series, treatment information, baseline vegetation characteristics, baseline soil nutrient data, relative enrichment-intensity variables and site coordinates.

These outputs are used as inputs for the subsequent scripts that calculate stability metrics and reproduce the statistical analyses, figures and tables in the manuscript.

Setup work environment

```
> # Start from a clean R session.
> rm(list = ls())
> gc()
>
```

```
> # Load the packages required for data cleaning, taxonomic harmonization,
> # community calculations, spatial data handling and visualization.
> Packages <- c(
+   "tidyverse", "magrittr", "grid", "gridExtra", "rnaturalearth", "rnaturalearthdata",
+   "ggcorrplot", "scales", "sp", "sf", "broom", "nlme", "scico", "vegan", "dlookr", "GGally",
+   "PerformanceAnalytics"
+ )
> pacman::p_load(char = Packages)
>
> # Use the dplyr versions of these functions explicitly because functions with
> # the same names can be loaded from other packages.
> select <- dplyr::select
> summarise <- dplyr::summarise
>
> # Define project paths. The script assumes that raw input files are stored in
> # "mydata" and that figures are written to "myfigure", both relative to the project root.
> main.path <- getwd()
> data.path <- paste0(main.path, "/mydata")
> figure.path <- paste0(main.path, "/myfigure")
>
> # Set a seed to make any random choices reproducible.
> set.seed(1234)
```

Section 1: Clean plant cover data

1.1 Read the raw plant cover data

The raw NutNet cover file contains annual species-level cover records for all available sites, treatments, plots and years.

```
> mydata <- read_csv(paste0(data.path, "/full-cover_2024-05-31.csv"))
```

1.2 Retain live plant cover records

Only live plant cover is used to quantify community composition and compositional stability. Records corresponding only to dead plant material are therefore removed.

```
> mydata <- filter(mydata, live == 1)
```

1.3 Check for missing treatment information

Treatment identity is required to distinguish control plots from nutrient-addition plots. This check confirms whether any cover records lack treatment information.

```
> any(is.na(mydata$trt))
```

1.4 Use `site_code` as the site identifier

The `site_name` column contains formatting inconsistencies across records. The standardized `site_code` field is therefore used as the site identifier throughout the workflow.

```
> mydata %<>% select(-site_name) # remove the site_name column
```

1.5 Require observations from treatment year 0 to treatment year 4

Stability metrics require repeated observations through time. Plots are excluded if any of the first five treatment-year records (`year_trt` 0, 1, 2, 3 or 4) are missing.

```
> plots.with.one.from.year0.to.year4.missing <- mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest() %>%
+   mutate(result = map_lgl(data, function(dat) {
+     length(setdiff(c(0, 1, 2, 3, 4), dat$year_trt)) > 0
```

```
+   })) %>%
+   filter(result) %>%
+   select(-data, -result)
>
> mydata <- anti_join(mydata, plots.with.one.from.year0.to.year4.missing)
> rm(plots.with.one.from.year0.to.year4.missing)
```

1.6 Remove observations after the first sampling gap

For each plot, the time series is truncated at the first gap in annual sampling. This ensures that subsequent stability calculations are based on uninterrupted annual observations.

```
> mydata2 <- mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest()
>
> mydata2 <- mydata2 %>%
+   mutate(data2 = map(data, function(df1) {
+     unique_years <- unique(df1$year_trt)
+     if (length(unique_years) == max(unique_years) + 1) {
+       df1 <- df1
+     } else {
+       gap_years <- setdiff(0:max(unique_years), unique_years)
+       df1 <- df1 %>% filter(year_trt < min(gap_years))
+     }
+     return(df1)
+   }))
>
> mydata <- mydata2 %>%
+   select(-data) %>%
+   unnest(data2) %>%
+   ungroup()
```

1.7 Check whether multiple subplots were sampled within plots

This diagnostic checks whether any plot contains records from more than one subplot, which would require additional aggregation before plot-level analyses.

```
> mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   summarise(n.subplot = length(unique(subplot))) %>%
+   ungroup() %>%
+   filter(n.subplot > 1)
```

No plot includes multiple sampled subplots in this dataset, so no additional aggregation is required at this step.

1.8 Remove blocks containing only one treatment

Blocks containing only a single treatment level including the control do not provide a useful within-block treatment comparison and are excluded.

```
> single.treatment.block <- mydata %>%
+   group_by(site_code, block) %>%
+   summarize(n.trt = n_distinct(trt)) %>%
+   ungroup() %>%
+   filter(n.trt == 1)
> mydata <- anti_join(mydata, single.treatment.block) # removed the relevant plots
> rm(single.treatment.block)
```

1.9 Remove blocks without a control plot

Because stability responses are quantified relative to unmanipulated controls, blocks without a control plot are excluded.

```
> no.control.block <- mydata %>%
+   group_by(site_code, block) %>%
+   summarize(control.check = any(unique(trt) == "Control")) %>%
+   ungroup() %>%
+   filter(!control.check)
> mydata %<>% anti_join(no.control.block)
```

1.10 Compile the cleaned cover dataset

The cleaned cover dataset is reduced to unique records and stored for subsequent taxonomic harmonization.

```
> mydata.coverage <- mydata %>%
+   droplevels() %>%
+   distinct()
> rm(mydata)
```

Section 2: Harmonize taxonomic names in the cover dataset

This section standardizes plant taxonomic names within each site before community-level stability metrics are calculated. Taxonomic harmonization is performed within sites because the main goal is to ensure temporal consistency of species identities within each local community.

2.1 Apply the first round of taxonomic standardization

The first harmonization step uses the existing NutNet taxonomic adjustment function provided by Yann Hautier.

```
> data1 <- mydata.coverage
> code.path <- paste0(main.path, "/other_code")
> source(paste0(code.path, "/Taxonomic.Adjustments.function.191115.R"))
> library(plyr)
> data1 <- Taxonomic.Adjustments(datafile = data1)
```

2.2 Remove unidentified taxa

Records labelled as `UNKNOWN` cannot be assigned consistently to species or genus-level entities and are removed before compositional analyses.

```
> data1$Taxon %>%
+   str_subset("UNKNOWN") %>%
+   unique()
> data1 %<>% filter(!str_detect(Taxon, "UNKNOWN"))
```

2.3 Aggregate taxa to genus level when unresolved species occur

When a site contains both identified species within a genus and an unresolved record of the same genus (`Genus SP.`), records from that genus are aggregated to `Genus SP.` within the site. This conservative step avoids treating potentially identical taxa as distinct entities.

```
> aggregate.to.genus.level.for.sp.f <- function(Site.code = "bldr.us") {
+   df <- data1 %>%
+     filter(site_code == Site.code)
+   df <- df %>%
+     mutate(splitted = map(
+       Taxon,
+       function(x) str_split(x, pattern = " ")[[1]]
+     )) %>%
+     mutate(
+       genus = map_chr(splitted, function(x) x[1]),
+       epithet = map_chr(splitted, function(x) x[2])
```

```
+     )
+   genus.to.aggregate <- df %>%
+     filter(epithet == "SP.") %>%
+     select(genus) %>%
+     distinct() %>%
+     unlist()
+   if (length(genus.to.aggregate) > 0) {
+     df1 <- df %>%
+       filter(genus %in% genus.to.aggregate)
+     df2 <- df %>%
+       filter(genus %in% setdiff(df$genus, genus.to.aggregate))
+     df1 %<>% mutate(Taxon = paste(genus, "SP."))
+     df <- df1 %>% bind_rows(df2)
+     df %<>% select(-splitted, -genus, -epithet)
+     df %<>%
+       dplyr::group_by(
+         site_code, block, plot, subplot,
+         trt, year, year_trt, Taxon
+       ) %>%
+       dplyr::summarise(max_cover = sum(max_cover)) %>%
+       ungroup()
+   } else {
+     df %<>%
+       select(-splitted, -genus, -epithet) %>%
+       as_tibble()
+   }
+   return(df)
+ }
>
> L <- data1 %>%
+   select(site_code) %>%
+   distinct() %>%
+   as_tibble() %>%
+   mutate(dat = map(site_code, aggregate.to.genus.level.for.sp.f))
> data1 <- L %>%
+   dplyr::rename(sites = site_code) %>%
+   unnest(cols = dat) %>%
+   select(-sites)
```

```
>
> mydata.coverage <- data1
> rm(data1)
> detach("package:plyr", unload = TRUE)
```

Section 3: Clean plant biomass data

3.1 Read the raw biomass data

The raw biomass file contains annual aboveground biomass measurements for NutNet plots.

```
> mydata <- read_csv(paste0(data.path, "/full-biomass_2024-05-31.csv"))
```

3.2 Retain live biomass records

Only live aboveground plant biomass is used to quantify functional stability. Records corresponding only to dead biomass are therefore removed.

```
> mydata <- filter(mydata, live == 1)
```

3.3 Check for missing treatment information

Treatment identity is required for pairing nutrient-addition plots with the corresponding control plots.

```
> any(is.na(mydata$trt))
```

3.4 Use `site_code` as the site identifier

As for the cover dataset, the standardized `site_code` field is used as the site identifier.

```
> mydata %<>% select(-site_name) # remove the site_name column
```

3.5 Require observations from treatment year 0 to treatment year 4

Plots are excluded if any of the first five treatment-year records (`year_trt` 0, 1, 2, 3 or 4) are missing.

```
> plots.with.one.from.year0.to.year4.missing <- mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest() %>%
+   mutate(result = map_lgl(data, function(dat) {
+     length(setdiff(c(0, 1, 2, 3, 4), dat$year_trt)) > 0
+   })) %>%
+   filter(result) %>%
+   select(-data, -result)
>
> mydata <- anti_join(mydata, plots.with.one.from.year0.to.year4.missing)
> rm(plots.with.one.from.year0.to.year4.missing)
```

3.6 Remove observations after the first sampling gap

For each plot, the biomass time series is truncated at the first gap in annual sampling so that stability calculations use continuous time series.

```
> mydata2 <- mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest()
>
> mydata2 <- mydata2 %>%
+   mutate(data2 = map(data, function(df1) {
+     unique_years <- unique(df1$year_trt)
+     if (length(unique_years) == max(unique_years) + 1) {
+       df1 <- df1
+     } else {
+       gap_years <- setdiff(0:max(unique_years), unique_years)
+       df1 <- df1 %>% filter(year_trt < min(gap_years))
+     }
+     return(df1)
+   }))
>
> mydata <- mydata2 %>%
+   select(-data) %>%
+   unnest(data2) %>%
+   ungroup()
```

3.7 Check whether multiple subplots were sampled within plots

This diagnostic checks whether additional aggregation is needed before the biomass data are used at the plot level.

```
> mydata %>%
+   group_by(site_code, block, trt, plot) %>%
+   summarise(n.subplot = length(unique(subplot))) %>%
+   ungroup() %>%
+   filter(n.subplot > 1)
```

No plot includes multiple sampled subplots in this dataset, so no additional aggregation is required at this step.

3.8 Remove blocks containing only one treatment

Blocks containing only one treatment level are excluded because they do not allow within-block treatment-control comparisons.

```
> single.treatment.block <- mydata %>%
+   group_by(site_code, block) %>%
+   summarize(n.trt = n_distinct(trt)) %>%
+   ungroup() %>%
+   filter(n.trt == 1)
> mydata <- anti_join(mydata, single.treatment.block) # removed the relevant plots
> rm(single.treatment.block)
```

3.9 Remove blocks without a control plot

Blocks without a control plot are excluded because functional stability responses are quantified relative to unmanipulated controls.

```
> no.control.block <- mydata %>%
+   group_by(site_code, block) %>%
+   summarize(control.check = any(unique(trt) == "Control")) %>%
+   ungroup() %>%
+   filter(!control.check)
> mydata %<>% anti_join(no.control.block)
```

3.10 Compile the cleaned biomass dataset

The cleaned biomass dataset is reduced to unique records and saved for merging with the cover dataset.

```
> mydata.biomass <- mydata %>%
+   droplevels() %>%
+   distinct()
```

Section 4: Merge cover and biomass data

The cleaned cover and biomass datasets are nested at the plot level and merged. This creates a single plot-level object containing the available community composition and biomass time series for each plot.

```
> mydata.coverage.nested <- mydata.coverage %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest() %>%
+   rename(coverage.data = data)
> mydata.biomass.nested <- mydata.biomass %>%
+   group_by(site_code, block, trt, plot) %>%
+   nest() %>%
+   rename(biomass.data = data)
> df.tidy <- mydata.coverage.nested %>%
+   full_join(mydata.biomass.nested)
> # Add an indicator describing whether each plot has cover data, biomass data, or both.
> df.tidy <- df.tidy %>%
+   mutate(data.availability = map2_chr(coverage.data, biomass.data, function(df1, df2) {
+     if (is.null(df1)) {
+       output <- "only biomass data"
+     } else
+     if (is.null(df2)) {
+       output <- "only coverage data"
+     } else {
+       output <- "both biomass and coverage data"
+     }
+     return(output)
+   }))
> df.tidy <- df.tidy %>%
+   ungroup()
```

Section 5: Add plot characteristics

This section adds baseline plot and site characteristics used in later analyses, including total cover, total biomass, species richness, Shannon diversity, Pielou evenness of the plots before nutrient addition, baseline soil nutrient concentrations, relative nutrient-enrichment intensity and geographic coordinates.

5.1 Calculate baseline vegetation characteristics

Baseline vegetation characteristics are calculated from treatment year 0 before the post-treatment stability windows are analysed.

```
> df.tidy <- df.tidy %>%
+   mutate(total.cover = map2_dbl(coverage.data, data.availability, function(dat, DA) {
+     if (DA == "only biomass data") {
+       out <- NA
+     } else {
+       out <- dat %>%
+         filter(year_trt == 0) %>%
+         pull(max_cover) %>%
+         sum()
+     }
+     return(out)
+   })) %>%
+   mutate(richness = map2_dbl(coverage.data, data.availability, function(dat, DA) {
+     if (DA == "only biomass data") {
+       out <- NA
+     } else {
+       out <- dat %>%
+         filter(year_trt == 0) %>%
+         pull(Taxon) %>%
+         n_distinct()
+     }
+     return(out)
+   })) %>%
+   mutate(shannon.diversity = map2_dbl(coverage.data, data.availability, function(dat, DA) {
+     if (DA == "only biomass data") {
+       out <- NA
+     } else {
+       out <- dat %>%
+         filter(year_trt == 0) %>%
+         pull(max_cover) %>%
+         vegan::diversity()
+     }
+     return(out)
+   })) %>%
+   mutate(total.mass = map2_dbl(biomass.data, data.availability, function(dat, DA) {
+     if (DA == "only coverage data") {
+       out <- NA
```

```
+     } else {
+       out <- dat %>%
+         filter(year_trt == 0) %>%
+         pull(mass) %>%
+         sum()
+     }
+     return(out)
+   })) %>%
+   mutate(pielou.evenness = shannon.diversity / log(richness))
> # Pielou evenness is undefined when richness equals one because log(1) = 0.
> # These single-species plots are assigned an evenness value of 0.
> df.tidy$pielou.evenness[df.tidy$richness == 1] <- 0
```

5.2 Add baseline soil nutrient concentrations and relative enrichment intensity

Baseline soil N, P and K measurements are merged into the plot-level dataset. These measurements are used to calculate the relative intensity of nutrient enrichment as the amount of nutrient added relative to the baseline soil nutrient pool.

```
> df.nutrient <-
+   read_csv(paste0(data.path, "/full-soil_2024-05-31.csv")) %>%
+   rename(trt = Treatment) %>%
+   select(site_code, block, plot, year_trt, trt, pct_N, ppm_P, ppm_K) %>%
+   filter(year_trt == 0)
> # Convert character-coded missing values to numeric NA values.
> replace.null.f <- function(x) {
+   x[x == "NULL"] <- NA
+   y <- as.numeric(x)
+   return(y)
+ }
> df.nutrient %<>% mutate_at(c("pct_N", "ppm_P", "ppm_K"), replace.null.f)
> # Remove blocks where one or more baseline nutrient variables are entirely missing.
> blocks.to.exclude <- df.nutrient %>%
+   group_by(site_code, block) %>%
+   nest() %>%
+   mutate(check_ = map_lgl(data, function(dat) {
+     all(is.na(dat$pct_N)) | all(is.na(dat$ppm_P)) | all(is.na(dat$ppm_K))
+   })) %>%
+   filter(check_) %>%
+   ungroup() %>%
```

```
+   select(site_code, block)
> df.nutrient %<>% anti_join(blocks.to.exclude)
>
> # Some plots lack individual nutrient measurements although other plots in the same block
> # have available values. Missing plot-level nutrient values are imputed using the block mean.
> df.nutrient %<>%
+   group_by(site_code, block) %>%
+   nest() %>%
+   mutate(data = map(data, imputeTS::na_mean)) %>%
+   unnest(cols = data) %>%
+   ungroup()
> # Add indicators for whether each treatment receives N, P and/or K.
> df.nutrient.addition <- data.frame(
+   trt = c("Control", "N", "P", "K", "NP", "NK", "PK", "NPK", "Fence", "NPK+Fence"),
+   N_addition = c(0, 1, 0, 0, 1, 1, 0, 1, 0, 1),
+   P_addition = c(0, 0, 1, 0, 1, 0, 1, 1, 0, 1),
+   K_addition = c(0, 0, 0, 1, 0, 1, 1, 1, 0, 1)
+ )
> df.nutrient <- left_join(df.nutrient, df.nutrient.addition)
> # Calculate relative enrichment intensity for N, P and K.
> df.nutrient %<>%
+   mutate(
+     N_added_proportion = N_addition * 10 / pct_N,
+     P_added_proportion = P_addition * 10 / ppm_P,
+     K_added_proportion = K_addition * 10 / ppm_K
+   )
> # Merge nutrient variables into the plot-level dataset.
> df.tidy %<>% inner_join(df.nutrient %>% select(-year_trt))
> # Diagnose missing values after merging nutrient data.
> df.tidy %>%
+   select(-coverage.data, -biomass.data) %>%
+   diagnose()
> # At this stage, NA values are expected only where plots have cover data but no biomass data,
> # or biomass data but no cover data.
```

5.3 Apply final cleaning criteria

Some blocks can lose all control plots after merging with soil data and applying the inclusion criteria. Such blocks are removed because treatment responses require a control comparison.

```
> no.control.block <- df.tidy %>%
+   group_by(site_code, block) %>%
+   summarize(control.check = any(unique(trt) == "Control")) %>%
+   filter(!control.check)
> df.tidy %<>%
+   anti_join(no.control.block, by = c("site_code", "block"))
```

For this project, the analytical dataset is restricted to unmanipulated controls and single-nutrient addition treatments: N, P and K. Multi-nutrient treatments are not used because relative enrichment intensity is analysed separately for each focal nutrient.

```
> df.tidy <- df.tidy %>%
+   filter(trt %in% c("Control", "N", "P", "K"))
```

It is possible to lose plots by the selection procedures above and result in sites with only one treatment including the control. Remove such sites

```
> df.sites.single.trt <- df.tidy %>%
+   group_by(site_code) %>%
+   summarize(n.trt = n_distinct(trt)) %>%
+   ungroup() %>%
+   filter(n.trt == 1)
> df.tidy %<>% anti_join(df.sites.single.trt, by = "site_code")
```

5.4 Add site coordinates

Site coordinates are merged into the plot-level dataset.

```
> all.site.location <- read_csv(paste0(data.path, "/sites_2023-11-07.csv"))
> # Confirm that all selected sites have location information.
> setdiff(df.tidy$site_code, all.site.location$site_code)
> df.tidy <- df.tidy %>%
+   left_join(all.site.location %>%
+     select(site_code, latitude, longitude))
> rm(all.site.location)
> save(df.tidy, file = paste0(data.path, "/df.tidy.Rdata"))
```